BRACT's

Vishwakarma Institute of Information Technology, Pune Practical Implementation Sheet

Department: CS-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No:1 Roll no: 21 PRN No: 12411682

Subject Name: Deep Learning

Name of Student: Pranay Zunzunwala

Problem Statement: Install and configure TensorFlow/Keras in Google Colab. Perform data preprocessing, normalization, train-test splitting, and data visualization on a sample dataset.

Algorithm:

Step 1: Import the required libraries such as NumPy, Matplotlib, Seaborn, TensorFlow/Keras, and the necessary modules from scikit-learn.

Step 2: Install and configure TensorFlow/Keras in Google Colab and verify the TensorFlow and Keras versions along with GPU availability.

Step 3: Load the **MNIST handwritten digit dataset** using Keras and separate it into training and testing images and their corresponding labels.

Step 4: Inspect the dataset by checking the dimensions, pixel-value range, unique labels, and displaying sample handwritten digit images.

Step 5: Preprocess the image data by converting the pixel values to float32 and **normalize the pixel values from 0–255 to 0–1**.

Step 6: Split the original training data into **80% training and 20% validation data** using train_test_split, while keeping the test dataset separate for final evaluation.

Step 7: Visualize the preprocessed data by displaying normalized images, the distribution of digit classes, and the distribution of normalized pixel intensities.

Step 8: Create a **Sequential Neural Network** using TensorFlo/Keras with a Flatten layer, Dense hidden layer using **ReLU**, Dropout layer, and an output Dense layer using **Softmax** activation.

Step 9: Compile and train the neural network using the **Adam optimizer**, sparse categorical cross-entropy loss, and accuracy as the evaluation metric.

Step 10: Evaluate the trained model using the test dataset and visualize the training and validation accuracy curves to analyze the performance of the model.

Code:

```
# Step 1: Install and Import Required Libraries

!pip -q install -U tensorflow keras matplotlib seaborn scikit-learn

import tensorflow as tf

from tensorflow import keras

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

# Display TensorFlow and Keras information

print("TensorFlow version:", tf.__version__)

print("Keras version:", keras.__version__)

print("GPU available:", tf.config.list_physical_devices('GPU'))

# Step 2: Load the MNIST Dataset

(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

print("\nOriginal training data shape:", x_train.shape)

print("Original training labels shape:", y_train.shape)

print("Original test data shape:", x_test.shape)

print("Original test labels shape:", y_test.shape)

# Step 3: Inspect the Dataset
```

```
print("\nPixel value range:", x_train.min(), "to", x_train.max())
print("Unique labels:", np.unique(y_train))
# Display 10 sample images
plt.figure(figsize=(10, 5))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i], cmap="gray")
    plt.title(f"Label: {y_train[i]}")
    plt.axis("off")
plt.tight_layout()
plt.show()
# Step 4: Data Preprocessing and Normalization
# Convert pixel values to float32
x_train = x_train.astype("float32")
x_test = x_test.astype("float32")
# Normalize pixel values from [0, 255] to [0, 1]
x_train = x_train / 255.0
x_test = x_test / 255.0
print("\nAfter normalization:")
print("Training pixel range:", x_train.min(), "to", x_train.max())
print("Test pixel range:", x_test.min(), "to", x_test.max())
# Step 5: Train-Test / Training-Validation Split
from sklearn.model_selection import train_test_split
```

```
# Split training data into 80% training and 20% validation
x_train_split, x_val, y_train_split, y_val = train_test_split(
    x_train,
    y_train,
    test_size=0.20,
    random_state=42,
    stratify=y_train
)
print("\nTraining set:", x_train_split.shape, y_train_split.shape)
print("Validation set:", x_val.shape, y_val.shape)
print("Test set:", x_test.shape, y_test.shape)
# Step 6: Visualize Normalized Images
plt.figure(figsize=(10, 5))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(
        x_train_split[i],
        cmap="gray",
        vmin=0,
        vmax=1
    )
    plt.title(f"Digit: {y_train_split[i]}")
    plt.axis("off")
```

```
plt.suptitle("Normalized MNIST Images")
plt.tight_layout()
plt.show()
# Step 7: Visualize Class Distribution
unique, counts = np.unique(y_train, return_counts=True)
plt.figure(figsize=(9, 5))
sns.barplot(
    x=unique,
    y=counts
)
plt.title("Distribution of Digit Classes in MNIST Training Data")
plt.xlabel("Digit")
plt.ylabel("Number of Images")
plt.show()
print("\nClass counts:")
for digit, count in zip(unique, counts):
    print(f"Digit {digit}: {count}")
# Step 8: Visualize Pixel Intensity Distribution
plt.figure(figsize=(9, 5))
plt.hist(
    x_train.flatten(),
    bins=50
)
```

```
plt.title("Distribution of Normalized Pixel Intensities")
plt.xlabel("Normalized Pixel Value")
plt.ylabel("Frequency")
plt.show()
# Step 9: Display One Image and Its Pixel Matrix
sample_index = 0
plt.figure(figsize=(6, 5))
plt.imshow(
    x_train_split[sample_index],
    cmap="gray",
    vmin=0,
    vmax=1
)
plt.colorbar(
    label="Normalized Pixel Value"
)
plt.title(
    f"Sample Image - Label: {y_train_split[sample_index]}"
)
plt.show()
print("\nImage shape:", x_train_split[sample_index].shape)
print("\nFirst 5 rows of normalized pixel values:")
print(x_train_split[sample_index][:5])
```

```
# Step 10: Create Sequential Neural Network
model = keras.Sequential([
    keras.layers.Input(shape=(28, 28)),
    # Convert 28x28 image into a 1D vector
    keras.layers.Flatten(),
    # Hidden layer
    keras.layers.Dense(
        128,
        activation="relu"
    ),
    # Dropout for regularization
    keras.layers.Dropout(0.2),

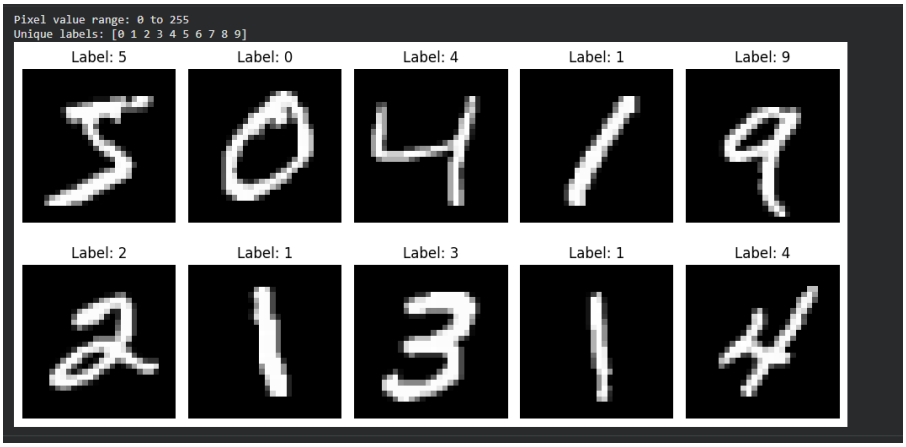
    # Output layer for 10 digit classes
    keras.layers.Dense(
        10,
        activation="softmax"
    )
])
# Compile the Model
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
```

```
    metrics=["accuracy"]
)
print("\nModel Summary:")
model.summary()
# Step 11: Train the Model
history = model.fit(
    x_train_split,
    y_train_split,
    validation_data=(x_val, y_val),
    epochs=3,
    batch_size=128,
    verbose=1
)

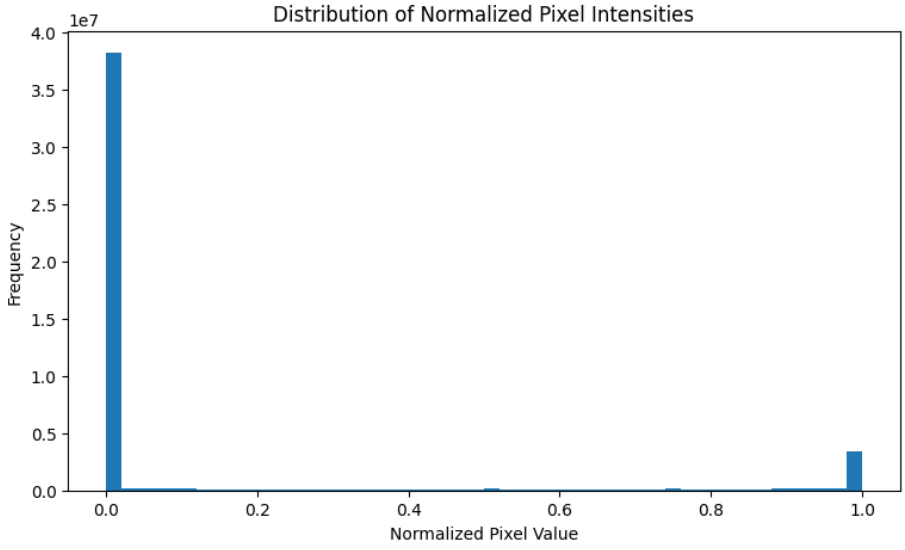
# Step 12: Visualize Training and Validation Accuracy
plt.figure(figsize=(9, 5))
plt.plot(
    history.history["accuracy"],
    marker="o",
    label="Training Accuracy"
)
plt.plot(
    history.history["val_accuracy"],
```

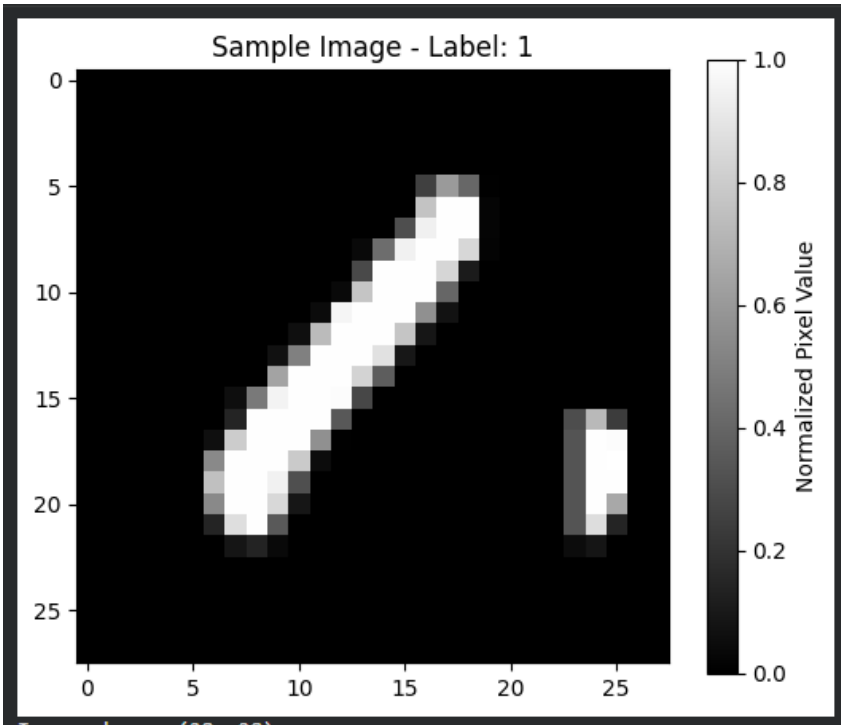
```
    marker="o",
    label="Validation Accuracy"
)
plt.title("Training and Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()
# Step 13: Evaluate Model on Test Dataset
test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)
print("\nFinal Model Evaluation")
print("----------------------")
print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")
```

Output:


Pixel value range: 0 to 255
Unique labels: [0 1 2 3 4 5 6 7 8 9]
Label: 5
Label: 0
Label: 4
Label: 1
Label: 9
Label: 2
Label: 1
Label: 3
Label: 1
Label: 4


Distribution of Digit Classes in MNIST Training Data
Number of Images
Digit
0
1000
2000
3000
4000
5000
6000
7000
0
1
2
3
4
5
6
7
8
9


Distribution of Normalized Pixel Intensities
1e7
Frequency
Normalized Pixel Value
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
0.0
0.2
0.4
0.6
0.8
1.0

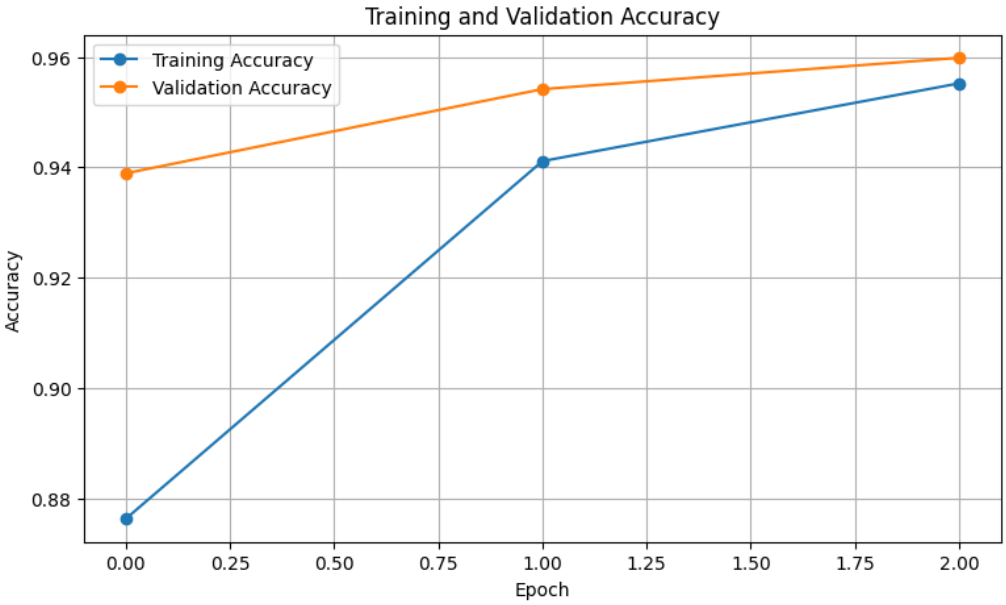



```
Image shape: (28, 28)
First 5 rows of normalized pixel values:
[[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
  0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
  0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
  0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
  0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
  0. 0. 0. 0.]]
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 101,770 (397.54 KB)
Trainable params: 101,770 (397.54 KB)
Non-trainable params: 0 (0.00 B)




```
Test Loss: 0.1238
Test Accuracy: 0.9639
Test Accuracy: 96.39%
```